React Hooks Cheatsheet

Quick reference for essential React Hooks with common patterns and usage examples.

useState

Local component state that persists between re-renders and triggers updates when changed.

Basic API

```
const [state, setState] = useState(initialState)
```

Immutable updates

Always create new objects and arrays instead of mutating state.

```
setForm({ ...form, name: 'Taylor' });
setTodos([...todos, newTodo]);
setTodos(todos.filter(t => t.id !== id));
```

Updater functions

Use functions for dependent updates to avoid stale state.

```
function Counter() {
  const [count, setCount] = useState(0);
  const increment = () => setCount(c => c + 1);
  return <button onClick={increment}>{count}</button>;
}
```

Deriving state

Calculate values during render instead of storing them in state.

```
function OrderSummary({ items }) {
  const [discount, setDiscount] = useState(0);
  const subtotal = items.reduce((sum, item) => sum +
item.price, 0);
  const total = subtotal - discount;
  return <div>Total: ${total}</div>;
}
```

State organization

Structure state to avoid contradictions and duplication. Group related state that changes together.

```
const [status, setStatus] = useState('idle');
const [position, setPosition] = useState({ x: 0, y: 0 });
```

Resetting state

Use `key` prop to reset component state when data changes.

```
<Chat key={person.id} contact={person} />
```

- **Updates batched** Multiple `setState` calls in events are batched automatically
- **Stale closures** Use updater functions to avoid stale state

useContext

Share data across component tree without manually passing props through every level.

Basic API

```
const value = useContext(SomeContext);
```

Sharing and updating data

Create context, provide values, and consume anywhere in component tree.

```
const ThemeContext = createContext(null);

function MyApp() {
  return (
    <ThemeContext value="dark">
      <Form />
    </ThemeContext>
  );
}

function Button() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Click me</button>;
}
```

- **No prop drilling** Skip intermediate components entirely
- **Single source** Context value comes from nearest `Provider` above

useEffect

Escape hatch to synchronize with external systems and perform side effects after render.

Basic API

```
useEffect(setup, dependencies?)
```

Event listeners

Subscribe to DOM events and clean up on unmount.

```
useEffect(() => {
  const handleResize = () => setSize({ width:
window.innerWidth, height: window.innerHeight });
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize',
handleResize);
}, []);
```

External connections

Connect to external systems and clean up when done.

```
useEffect(() => {
  const connection = createConnection(roomId);
  connection.connect();
  return () => connection.disconnect();
}, [roomId]);
```

- **After render** Effects run after DOM updates, not during
- **Cleanup required** Always cleanup subscriptions and timers